

I n t e r p r e t e r

Interpreter Pattern

Evaluate a little language

DEFINITION

Given a language, define a representation for its grammar plus an interpreter to evaluate sentences.

Interpreter models a small language as a class hierarchy — one Expression class per grammar rule — so a sentence parses into a tree of these objects (an AST that is itself a Composite). Each node knows how to `interpret(context)`, recursing into its children, so evaluating the root walks the whole tree.

WHY IT MATTERS

For a small, stable grammar that recurs — a filter expression, a business rule, a search query — hand-coding ad-hoc parsing scatters the logic and resists change. Mapping each rule to a class makes the grammar explicit and easy to extend: a new rule is a new class. The catch is scale — complex grammars explode into too many classes, and tree-walking is slow.

— How it works

AbstractExpression Declares `interpret(context)` for every node in the grammar.

TerminalExpression A leaf — a literal or variable, the atoms of the grammar.

NonterminalExpression A composite rule (And, Or, Plus) holding sub-expressions.

HOW TO APPLY

1. Confirm the grammar is small and stable enough to hand-model.
2. Give every rule an Expression class with one `interpret(ctx)` method.
3. Make terminals leaves and nonterminals hold child expressions (a Composite).
4. Parse text into the tree separately; then interpret the root to evaluate.

LITMUS TEST

Is this a small, stable grammar you evaluate often — and would one class per rule stay manageable? If the grammar is large or speed matters, reach for a parser generator instead.

— Smell → fix

A growing if/else hand-evaluates every rule kind in one tangled function:

```
// one function hand-evaluates every rule kind
if (e.op === 'var') return ctx[e.name]
if (e.op === 'and') return ev(e.l) && ev(e.r)
// a new rule edits this function again
```

↓ ONE CLASS PER GRAMMAR RULE

```
interface Expr { interpret(c): boolean }
const Var = (n): Expr => ({ interpret: c => c[n] })
const And = (l: Expr, r: Expr): Expr => ({
  interpret: c => l.interpret(c) && r.interpret(c)
})
// "adult AND admin" → And(Var('adult'), Var('admin'))
tree.interpret(ctx) // walks the AST
```

Each rule is now a small, local `interpret()` — a new rule is a new class, not another branch. The AST is a Composite; add operations (print, type-check) with a Visitor instead of bloating the nodes.

USE WHEN

- ✓ Small DSLs
- ✓ Rule engines; query/filter expressions

AVOID WHEN

- ▲ Complex grammars — use a real parser / AST tool

— Trade-offs & pitfalls

PROS

- + Grammar maps directly to classes
- + Easy to extend rules

CONS

- A class per rule explodes
- Poor performance for big grammars

COMMON PITFALLS

- ▲ Class-per-rule explodes: past a small, stable grammar it becomes unmanageable — use a parser generator (ANTLR, PEG).
- ▲ Parsing is a separate job: Interpreter defines and evaluates the AST; building it from text is the parser's responsibility.
- ▲ Tree-walking is slow — compile or cache hot expressions if performance matters.

WHERE YOU SEE IT

- ▶ Regular-expression matchers — a pattern compiled to an evaluable tree (the classic interpreter).
- ▶ Rules / specification engines: business rules as composable boolean expressions.
- ▶ SQL WHERE clauses and template / DSL engines (Liquid, SpEL) evaluating expression nodes.

SEE ALSO

Composite	the AST is a Composite — terminals are leaves, rules are nodes with children
Visitor	adds new operations over the tree without changing the node classes
Iterator	traverses the expression tree node by node
Strategy	a lighter alternative for pluggable rules when a full grammar is overkill

— Interview & sources

When is Interpreter appropriate?

Small, stable, well-understood grammars where a class-per-rule tree stays maintainable and speed isn't critical.

Interpreter vs a parser generator?

Interpreter hand-models and evaluates a tiny grammar; ANTLR / PEG generate efficient parsers for real languages.

How does it relate to Composite and Visitor?

The AST is a Composite; Visitor adds new operations over the nodes without changing them.

A real-world example?

Regular-expression matchers and rules / specification engines are interpreters of a small grammar.

REMEMBER

One class per grammar rule; a sentence becomes a tree you evaluate by walking it.

SOURCES

GoF — "Design Patterns" (1994): Interpreter (behavioral)

Martin Fowler — "Domain-Specific Languages" (2010): semantic model & tree-walking interpreters